

Lab 11: Store State Remotely on Terraform Cloud

Durée : 20 minutes

Cet atelier montre comment stocker l'état sur Terraform Cloud et lire à partir de celui-ci. Vous allez configurer un nouveau projet en utilisant Terraform Cloud comme support et utiliser un deuxième projet pour le lire.

- Tâche 1 : créer un User Token Terraform Cloud
- Tâche 2 : Créer une configuration qui stocke son état sur Terraform Cloud
- Tâche 3 : Créer une autre configuration Terraform qui lit à partir de l'état sur Terraform Cloud

Conditions préalables

Pour cet atelier, nous supposons que vous avez installé [Terraform](#) et que vous vous êtes [inscrit](#) pour un compte Terraform Cloud.

Tâche 1 : Créer un User Token Terraform Cloud

Afin de stocker l'état à distance sur Terraform Cloud, nous devons créer un jeton utilisateur et configurer notre environnement local pour utiliser ce jeton.

REMARQUE : Terraform Cloud ne fonctionne qu'avec la version 0.11.13 ou ultérieure.

Étape 1.1 :

Remarque : Vous pouvez ignorer cette étape si vous avez déjà créé une organisation.

[Connectez-vous](#) à Terraform Cloud et accédez à la nouvelle page de l'organisation :

- Les nouveaux utilisateurs sont automatiquement redirigés vers la nouvelle page de l'organisation.
- Si votre compte utilisateur est déjà membre d'une organisation, ouvrez le menu de changement d'organisation dans la barre de navigation supérieure et cliquez sur le bouton `Create new organization`.

Entrez un nom d'organisation unique et une adresse e-mail pour les notifications, puis cliquez sur le bouton `Create new organization`.

New Organization

ORGANIZATION NAME

This will be part of your resource names used in various tools, i.e. `hashicorp/www-prod`.

EMAIL ADDRESS

The organization email is used for any future notifications, such as billing, and the organization avatar, via gravatar.com.

Create organization

Étape 1.2 :

La CLI de Terraform a besoin d'informations d'identification avant de pouvoir accéder à Terraform Cloud. Nous allons d'abord créer le fichier de configuration Terraform CLI:

```
vi ~/.terraformrc
```

Dans ce fichier, nous ajouterons le bloc d'informations d'identification suivant :

```
credentials "app.terraform.io" {  
  token = "REPLACE_ME"  
}
```

Laissez votre éditeur ouvert.

Étape 1.3

Dans votre navigateur Web, rendez-vous dans la [section Tokens](#) de vos paramètres utilisateur ou cliquez sur l'icône de l'utilisateur dans le coin supérieur droit, cliquez sur "User Settings", puis cliquez sur "Tokens" dans la barre latérale gauche.

Générez un nouveau jeton en saisissant une description et en cliquant sur le bouton Create API token. Le nouveau jeton apparaîtra dans une zone de texte sous le champ de description.

Copiez le jeton dans le presse-papiers.

Dans votre éditeur de texte, collez le jeton dans l'argument **token**, en remplaçant l'espace réservé "REPLACE_ME". Enregistrez le fichier de configuration CLI et fermez votre éditeur.

Étape 1.4

À ce stade, Terraform peut utiliser Terraform Cloud avec n'importe quelle configuration Terraform qui a activé le backend `remote`.

Mettez à jour les autorisations du fichier de configuration CLI sur "0600" avec :

```
chmod 0600 ~/.terraformrc
```

Tâche 2 : Créer une configuration qui stocke son état sur Terraform Cloud

Pour cette tâche, vous allez créer un projet Terraform qui stocke son état dans Terraform Cloud et émet un output.

Étape 2.1

Dans cette étape, vous allez créer le projet et une configuration.

```
mkdir -p lab11/write_state && cd $_  
  
vi main.tf
```

Step 2.2.2

Configurez la configuration pour utiliser le backend `remote`, en remplaçant `ORGANIZATION NAME` par le nom de votre organisation :

```
# write_state/main.tf  
terraform {  
  backend "remote" {  
    hostname = "app.terraform.io"  
    organization = "<ORGANIZATION NAME>"  
  
    workspaces {  
      name = "lab_11_write_state"  
    }  
  }  
}
```

Étape 2.2.3

Ensuite, ajoutez la possibilité de générer et d'émettre une sortie `random` à partir de votre configuration :

```
# write_state/main.tf  
resource "random_id" "random" {  
  keepers = {  
    uuid = uuid()  
  }  
  
  byte_length = 8  
}  
  
output "random" {  
  value = random_id.random.hex  
}
```

Step 2.4

Provisionnez la ressource et transférez l'état vers Terraform Cloud avec :

```
terraform init  
  
terraform apply -auto-approve
```

Vous verrez Terraform confirmer qu'il crée votre état à distance ainsi que votre sortie "random". Si vous revenez à votre organisation, vous verrez également un nouveau nom d'espace de travail "lab_11_write_state".

Étape 2.2.5

Toutes nos félicitations! Vous stockez maintenant l'état à distance. Avec Terraform Cloud, vous pouvez partager votre espace de travail avec vos coéquipiers. De retour dans l'interface utilisateur de Terraform Cloud, vous pourrez :

- Afficher tous les espaces de travail de votre organisation
- Verrouillez un espace de travail, ce qui permet d'éviter facilement les modifications conflictuelles et la corruption d'état
- Afficher l'historique de l'état

Tâche 3 : créer une autre configuration Terraform qui lit à partir de l'état sur Terraform Cloud

Maintenant que notre état est stocké dans Terraform Cloud dans notre espace de travail lab_11_write_state, nous allons créer un autre projet, une configuration et un espace de travail à lire à partir de celui-ci.

Étape 3.1

Commencez par créer un nouveau répertoire et un nouveau fichier main.tf :

```
mkdir -p lab11/read_state && cd $_  
  
vi main.tf
```

Étape 3.2

Tout comme nous l'avons fait à l'étape 2.2, nous devons configurer notre configuration pour utiliser le backend remote, en remplaçant à nouveau ORGANIZATION NAME. Nous allons également créer une nouvelle ressource random à comparer :

```
# read_state/main.tf  
terraform {  
  backend "remote" {  
    hostname = "app.terraform.io"  
    organization = "<ORGANIZATION NAME>"
```

```

        workspaces {
            name = "lab_11_read_state"
        }
    }
}

resource "random_id" "random" {
    keepers = {
        uuid = uuid()
    }

    byte_length = 8
}

```

Étape 3.3

Afin de lire à partir de notre espace de travail `lab_11_write_state`, nous devons configurer une source de données `terraform_remote_state`. Les sources de données sont utilisées pour récupérer des données en lecture seule à partir de sources extérieures à notre projet. Il prend en charge plusieurs fournisseurs de cloud, mais nous utiliserons `remote` comme backend.

```

# read_state/main.tf
data "terraform_remote_state" "write_state" {
    backend = "remote"

    config = {
        hostname = "app.terraform.io"
        organization = "<ORGANIZATION NAME>"

        workspaces = {
            name = "lab_2_write_state"
        }
    }
}

```

Étape 3.4

Maintenant que nous avons accès à notre espace de travail distant `lab_11_write_state`, nous pouvons récupérer la sortie `random` qu'il contient. Nous afficherons également `random` que nous avons créé dans cette configuration, confirmant qu'ils sont distincts.

```

# read_state/main.tf
output "random" {
    value = random_id.random.hex
}

output "write_state_random" {
    value = data.terraform_remote_state.write_state.outputs.random
}

```

Étape 3.5

Pour vérifier que nous avons récupéré avec succès l'état de notre espace de travail `lab_11_write_state`, nous pouvons exécuter notre configuration et valider nos sorties.

Exécutez à nouveau `init` pour installer les fichiers de support nécessaires.

```
terraform init
```

Exécutez `apply` pour voir la sortie.

```
terraform apply -auto-approve
```

```
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

```
Outputs:
```

```
random = c1597ca0fbba3997
```

```
write_state_random = 0de9168d0b78ead6
```

Vous avez maintenant stocké avec succès vos états à distance et lu à partir de ces états distants.